

## TEMA 2

# BBDD2 Parcial

1era fecha 10/06/2026

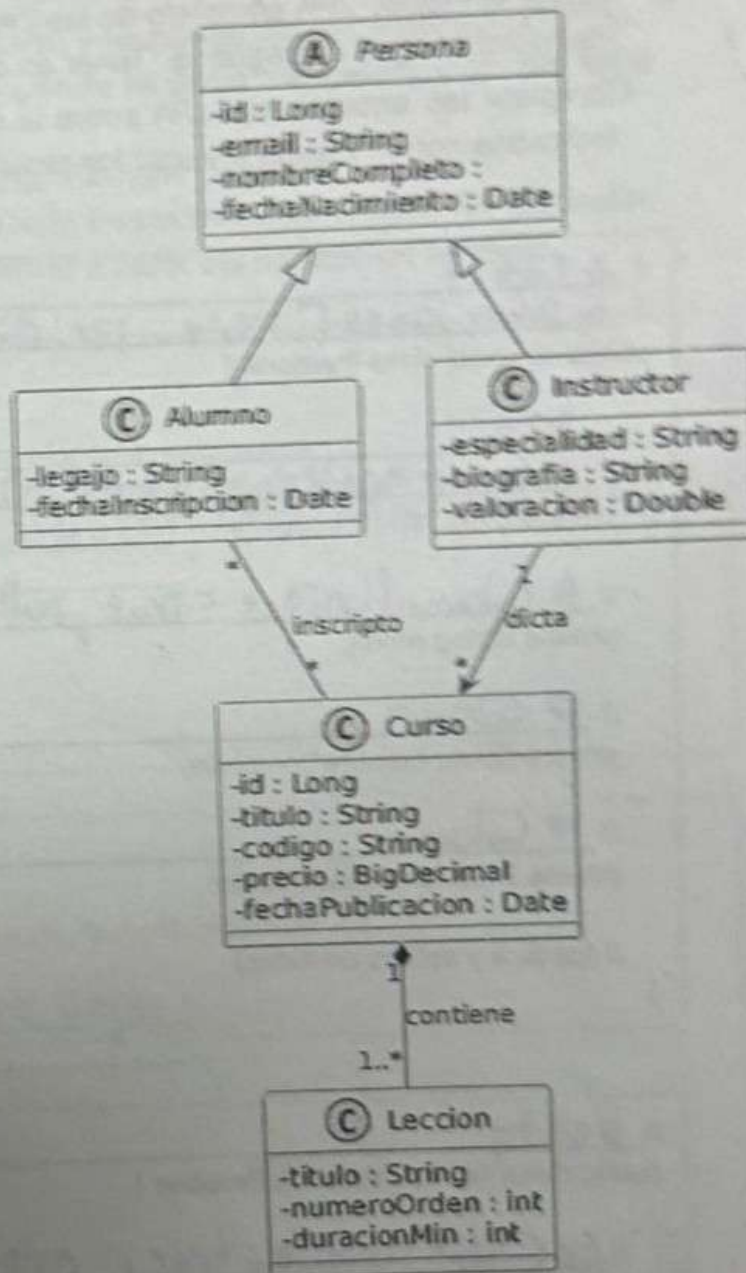
Uno interno de la cátedra

|   |   |   |   |   |   |   |   |    |   |
|---|---|---|---|---|---|---|---|----|---|
| 1 | 1 | 2 | 1 | 3 | 1 | 4 | 1 | 5  | 1 |
| 6 | 1 | 7 | 1 | 8 | 0 | 9 | 1 | 10 | 1 |

Corrigió: IVAN

El siguiente diagrama modela una plataforma de cursos online. Persona es una clase abstracta de la que heredan Alumno e Instructor. Un Alumno puede inscribirse en muchos Cursos y un Curso puede tener muchos alumnos inscritos (relación "inscripto", muchos-a-muchos). Un Instructor "dicta" muchos cursos, mientras que cada Curso es dictado por un único instructor (relación dicta, uno-a-muchos). Cada curso se compone de una o más lecciones (relación "contiene", composición).

Un alumno no puede ser instructor a la vez y viceversa y que existen pocas personas sobre todas las personas.



Persona (abstracta): id: Long, email: String (no puede repetirse para distintas personas), nombreCompleto: String, fechaNacimiento: Date

Alumno: legajo: String (no puede repetirse para distintos alumnos), fechaInscripcion: Date

Instructor: especialidad: String, biografia: String, valoracion: Double

Curso: id: Long, titulo: String, codigo: String (no puede repetirse para distintos cursos), precio: BigDecimal, fechaPublicacion: Date

Leccion: titulo: String, numeroOrden: int, duracionMin: int

## Hibernate / JPA

1. Al mapear la jerarquía Persona / Alumno / Instructor a tablas relacionales. ¿Cuál estrategia de herencia de las posibles opciones de Hibernate/JPA utilizaría? Justifica la decisión teniendo en cuenta que las subclases poseen atributos obligatorios propios ejemplo legajo en Alumno y especialidad en Instructor) y relaciones propias. Seleccione la única opción correcta:

- a. ☐ SINGLE\_TABLE
- b. ☐ JOINED
- c. ☒ TABLE\_PER\_CLASS
- d. ☐ No es posible mapear esta jerarquía con JPA porque las subclases tienen relaciones propias; debe eliminarse la herencia y duplicar los atributos comunes.

2. Definir el mapeo JPA completo de las tres clases de la jerarquía (Persona, Alumno e Instructor) según el diagrama. Tener en cuenta las restricciones de los atributos. Complete las anotaciones JPA sobre la siguiente estructura de clases, en los renglones indicados con //. Puede agregar los renglones que necesite.

// @Entity

// @Inheritance(strategy = GenerationType.TABLE)

public abstract class Persona {

// @Id

// @GeneratedValue(strategy = GenerationType.IDENTITY)

private Long id;

// @Column(unique = true, nullable = false)

private String email;

// @Column

private String nombreCompleto;

// @Column

private Date fechaNacimiento;

// (getters y setters omitidos)

// @Entity

public class Alumno extends Persona {

// @Column(unique = true, nullable = false)

private String legajo;

// @Column

private Date fechaInscripcion;



// @Entity

```
public class Instructor extends Persona {
```

// @Column

```
private String especialidad;
```

// @Column

```
private String biografia;
```

// @Column

```
private Double valoracion;
```

```
}
```

3. Analizar la relación 'dicta' entre Instructor y Curso y responda las siguientes preguntas:
- ¿Qué anotaciones se utilizan en cada lado? ¿Dónde se ubica mappedBy y en qué tabla aparece la clave foránea?
  - ¿Qué FetchType elegiría en cada extremo de la relación? Justificar.
  - ¿Qué operaciones en cascada configuraría desde Instructor hacia Curso? En particular, ¿tiene sentido CascadeType.REMOVE? Justificar a partir del modelo de negocio.

```
public class Instructor extends Persona {
```

```
// ... atributos propios ya mapeados (especialidad, biografia, valoracion)
```

```
// @OneToMany (mappedBy = "instructor", fetch = FetchType.LAZY)
```

```
private List<Curso> cursos = new ArrayList<>();
```

```
// getters y setters
```

```
}
```

```
public class Curso {
```

```
// ... atributos ya mapeados (id, titulo, codigo, precio, fechaPublicacion)
```

```
// @ManyToOne (fetch = FetchType.EAGER)
```

```
// @JoinColumn ("instructor")
```

```
private Instructor instructor;
```

```
// getters y setters
```

```
}
```

## Spring Data JPA

4. ¿Qué es Spring Data JPA y qué problema resuelve respecto de usar Hibernate directamente?

Seleccione todas las opciones correctas (puede haber más de una):



- a. ☒ Es una capa de abstracción sobre JPA/Hibernate que reduce el código repetitivo (boilerplate) de la capa de acceso a datos.
- b. ☒ Genera automáticamente, en tiempo de ejecución, la implementación de los repositorios definidos como interfaces.
- c. ☒ Permite derivar consultas a partir del nombre del método.
- d. ☐ Reemplaza por completo a Hibernate: al usar Spring Data JPA ya no interviene el ORM por debajo.
- e. ☒ Provee soporte de paginación y ordenamiento mediante Pageable y Sort.
- f. ☐ Es el componente encargado de generar el SQL y de mapear el ResultSet a objetos Java por sí mismo, sin intervención del ORM.
5. Describir dos situaciones/implementaciones concretas de este modelo donde Spring Data JPA simplifica código en comparación con la implementación manual requerida en Hibernate puro.
6. Definir la cabecera (interfaz) de repositorio de una de las tres entidades mapeadas.

// Repositorio de una de las entidades mapeadas (Persona, Alumno o Instructor)  
 public interface InstructorRepo extends JpaRepository<Instructor, Integer> {

// Método(s) de consulta (punto siguiente):

// @Query("Select i. curso from instructor i where i. valoración >: Val")  
List<Curso> buscarCursosMayorA(@param("Val") Integer valoración);  
 }

7. Sobre ese repositorio, agregar un método que devuelva los cursos dictados por instructores cuya valoración sea mayor a un valor dado. Indicar y justificar la estrategia elegida para escribir la consulta.

## MongoDB

8. Suponiendo una colección "cursos" en la que cada documento referencia a su instructor mediante un campo "instructorId", resolver con Aggregation Framework: por cada instructor, calcular la cantidad de cursos que dicta y el precio promedio de esos cursos; devolver únicamente los instructores con valoración mayor a 8, ordenados por precio promedio de forma descendente.
9. Suponiendo que este modelo se va persistir a MongoDB, seleccionar una relación del modelo que convenga modelar como documento embebido y otra que convenga modelar como referencia. Justificar cada una de las elecciones.

## Redis

10. ¿Dónde almacena Redis los datos? ¿Qué implicancias tiene esto en términos de velocidad y de persistencia? ¿Es posible persistir estos datos? Explique al menos una de las posibles estrategias.



1) Yo usaria la estrategia table-per-class ya que mantiene la integridad referencial de alumno con curso, e instructor con curso y ademas el padre no se relaciona con ningun otra entidad, siendo rapido esta estrategia al consultar una de las tablas de los hijos al no necesitar join con un padre, y ademas se menciona que hay pocas consultas sobre todas las personas, asi que no hay drama con la consulta polimorfica padre

3) a) Instructor tendra @OneToMany, y curso omento one, mappedBy va en el lado del instructor (donde no va la referencia) y la clase forenea va en curso, y joinedColumn en el lado con la referencia (curso)

b) Del lado del instructor Lazy, ya que si uno se trae todos los instructores, si tenemos eager, se traeria todos los cursos, lo que seria una consulta muy pesada inicial, y potencialmente consumiria mucha memoria

d) El lado del curso podria eager ya que es solo 1 objeto relacionado, y potencialmente es algo que se consulta junto (curso - instructor) tipo ukey

c) Ninguna / los instructores y cursos se modifican y crean separados en una plataforma normal, por tanto persist, merge y demás no van, remove tampoco porque ninguno seria dueño de la relacion segun el Uml, podrian existir por separado.



Como referencia pondría alumno - curso ya que alumno o curso podría cambiar y de tenerlo que modificarlo en múltiples lugares es problemático. Además la cantidad de personas por curso podría ser mucha como para embeberlo. Si este embebido

5) En el caso curso tener un repositorio te ahorra tener que crear a mano todas las operaciones de crud si se hereda de el menos Crud Repository y para cualquier consulta cumple como Instructor - curso no hay que instanciar la sesión ni mandarle mensajes, esto lo hace por nosotros Spring data y por lo cual lo hace más limpio y transparente para el programador porque el código se genera por el. ✓

7) Use @Query para hacer uso de HQL y su lenguaje que entiende y permite navegar objetos facilitado la consulta sin poner joins explícitos.

- 9) ✓ mi selección para el embedding es cursos - lección ya que una lección es de 1 solo curso y un curso tiene n lecciones, si se cambia 1 lección no hay que cambiar múltiples por redundancias y eliminar operación localización de registro a esta relación que puede ser muy consultada en conjunto, + hay tantos cursos como para moverlo y referenciar \*1
- 10) ✓ Redis almacena en RAM, lo cual lo hace rápido en consultas pero poco espacio y es volátil, se pueden persistir datos con RDB snapshots o AOF.

RDB es bajar los datos a memoria persistente cada x tiempo, a cambio de que algunos datos se pueden perder en este intervalo si es que la db se cae en ese momento y haya cambios no guardados

8) db. cursor.aggregate ([  
 \$lookup: { instructor, id, instructor\_id, instructor }  
 \$group: { instructor }  
 \$findMany: { \$gt: { instructor.vibracion },  
 \$project: { instructor.id, \$count: { instructor.cursos }, \$avg: { instructor.cursos.vibracion } } ] )